WEB PROGRAMMING I · LECTURE 4

CSS BOX MODEL AND UNITS

How elements occupy space on the web

A QUESTION

WHY DOES SPACING IN CSS SOMETIMES BEHAVE UNEXPECTEDLY?

You set a `width`. You add `padding`.
Suddenly your layout overflows.

PUZZLE

HOW WIDE IS THIS ELEMENT?

```
.box {
  width:   200px;
  padding:  20px;
  border:    2px solid white;
}
```

244px 200 + (20×2) + (2×2)

This is default browser behavior — and it catches almost every beginner.

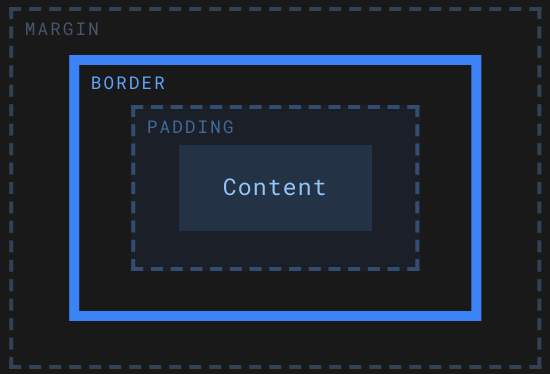
Everything on the web
is a box.

SECTION 1

THE CSS BOX MODEL

Four layers that control how every element occupies space

THE FOUR LAYERS



Every HTML element has all four layers — even when some are zero.

LAYER 1

CONTENT

- The actual text, image, or child elements
- `width` and `height` define the content area
- Background color fills from content outward through padding

```
.box {
  width:  300px;
  height: 150px;
  background: #1e3a5f;
}
```

LAYER 2

PADDING

- Space between the content and the border
- Transparent — shows the background color
- Increases the clickable / touch area

```
.box {
  padding: 16px;            /* all four sides */
  padding: 16px 24px;       /* top/bottom, left/right */
  padding: 8px 16px 8px 16px; /* top right bottom left */

  padding-top:    8px;
  padding-right:  16px;
  padding-bottom: 8px;
  padding-left:   16px;
}
```

LAYER 3

BORDER

- The visible edge of the element
- Three sub-properties: width · style · color
- By default, adds to the total rendered size

```
.box {
  /* shorthand */
  border: 2px solid #60a5fa;

  /* individual properties */
  border-width: 2px;
  border-style: solid;   /* dashed, dotted… */
  border-color: #60a5fa;

  /* one side only */
  border-bottom: 1px solid #334155;
}
```

LAYER 4

MARGIN

- Space *outside* the element
- Always transparent — no background color
- Separates elements from each other
- `margin: 0 auto` centers a block element

```
.box {
  margin: 16px;          /* all sides */
  margin: 16px auto;     /* top/bottom, center horizontally
  margin: 0 0 24px 0;   /* top right bottom left */

  /* centering pattern */
  width: 600px;
  margin: 0 auto;
}
```

PUTTING IT TOGETHER

A COMPLETE BOX

```
.card {
  /* Content dimensions */
  width:  300px;
  height: 120px;

  /* Padding */
  padding: 24px;

  /* Border */
  border: 2px solid #3b82f6;

  /* Margin */
  margin: 16px auto;

  background: #0f172a;
}
```

Total rendered width:

content	300px
padding (24 × 2)	48px
border (2 × 2)	4px
Total	**352px**

Margin is external — it is not part of the element's width.

SECTION 2

BOX SIZING

Controlling how `width` and `height` are calculated

THE PROBLEM

PADDING ADDS TO THE TOTAL WIDTH

Default behavior: `width: 200px` sets the *content* width only.
Padding and border are added on top.

```
.sidebar {
  width:   200px;  /* content only   → 200px */
  padding:  20px;  /* + 20 + 20      → 240px */
  border:    2px solid white;  /* + 2 + 2 → 244px */
}

/* Actual rendered width: 244px — not 200px! */
```

BOX-SIZING

TWO MODELS, TWO CALCULATIONS

CONTENT-BOX (DEFAULT)

`width` = content area only

BORDER-BOX

`width` = content + padding + border

With `border-box`, `width: 200px` always renders as exactly 200px. No surprises.

BEST PRACTICE

USE BORDER-BOX EVERYWHERE

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Why? The width you write is the width you get.

Most CSS frameworks and modern resets include this rule.

Put it at the very top of your stylesheet — **always.**

SECTION 3

CSS UNITS

How sizes are defined in CSS

TWO CATEGORIES

ABSOLUTE

Fixed size — does not change based on context

- `px` — pixels

RELATIVE

Scales based on another value

- `rem` — root font size
- `em` — parent font size
- `%` — parent dimension
- `vh / vw` — viewport

ABSOLUTE

PX — PIXELS

- Fixed size — does not scale with context
- Good for: borders, shadows, icon sizes
- Not ideal for: font sizes, responsive layouts

```
.card {
  width:  320px;
  height: 200px;

  border:        1px solid #334155;
  border-radius: 4px;

  font-size: 16px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.4);
}
```

On high-DPI (Retina) displays, the browser scales automatically — 1px still looks sharp.

RELATIVE

REM — ROOT EM

- Relative to the `<html>` font size
- Default: `1rem = 16px`
- Changing the root size scales everything proportionally
- Best practice for: font sizes, spacing

```
/* html default: font-size = 16px */

h1    { font-size: 2rem;    }  /* 32px */
h2    { font-size: 1.5rem;  }  /* 24px */
p     { font-size: 1rem;    }  /* 16px */
small { font-size: 0.75rem; }  /* 12px */

.section {
  padding: 2rem;    /* 32px */
  gap:     1.5rem;  /* 24px */
}
```

RELATIVE

EM — PARENT EM

- Relative to the *parent element's* font size
- Values can compound when nested
- Useful for component-local spacing
- Can get confusing in deeply nested elements

```
.parent {
  font-size: 20px;
}

.child {
  font-size: 1.5em;  /* 20 × 1.5 = 30px */
  padding:   1em;    /* 30 × 1.0 = 30px */
}

.grandchild {
  font-size: 1.5em;  /* 30 × 1.5 = 45px! */
  /* values multiply — watch out */
}
```

RELATIVE

% AND VH / VW

Percentage (%)

- Relative to the parent element's dimension
- `width: 50%` = half of parent's width

```
.container { width: 800px; }
.half      { width:  50%; } /* 400px */
```

Viewport units

- `vh` = 1% of viewport height
- `vw` = 1% of viewport width
- Independent of any parent element

```
.hero {
  height: 100vh; /* full screen height */
  width:  100vw; /* full screen width  */
}
```

QUICK REFERENCE

UNIT	TYPE	RELATIVE TO	COMMON USE
`px`	absolute	—	borders, shadows
`rem`	relative	root font-size	font sizes, spacing
`em`	relative	parent font-size	component-local spacing
`%`	relative	parent dimension	fluid widths
`vh`	relative	viewport height	hero sections
`vw`	relative	viewport width	full-bleed layouts

SECTION 4

DISPLAY PROPERTY

How elements flow on the page

DISPLAY

`block`

- Takes the full available width
- Always starts on a new line
- Respects `width` and `height`
- Examples: `div`, `p`, `h1`, `section`

```
div (block)
p (block)
h2 (block)
```

```
div {
  display: block; /* default */
}
```

DISPLAY

`inline`

- Only as wide as its content
- Flows with surrounding text — no line break
- `width` and `height` have *no effect*
- Examples: `span`, `a`, `strong`, `em`

```
Text with [span] and more [span] flowing inline.
```

```
span {
  display: inline; /* default */

  /* These have no effect on inline: */
  width:  200px;   /* ignored */
  height:  50px;   /* ignored */
}
```

DISPLAY

inline-block

- Flows inline — no forced line break
- But accepts `width`, `height`, padding, margin
- Best of both worlds
- Useful for: buttons, badges, nav items, cards

Box A Box B Box C

```
.item {
  display: inline-block;
  width:   80px;
  height:  40px;
  padding: 8px 12px;
}
```

DISPLAY

none

- Removes the element from the layout entirely
- Invisible and takes up *no space*
- Different from `visibility: hidden`

```
.box {
  display: none;
  /* Element is gone — no space taken */
}

.box-hidden {
  visibility: hidden;
  /* Invisible — but space is preserved */
}

/* Common pattern: toggle a menu */
.menu         { display: none; }
.menu.is-open { display: block; }
```

DISPLAY — AT A GLANCE

VALUE	LINE BREAK?	WIDTH / HEIGHT?	DEFAULT FOR
`block`	Yes — full width	Yes	div, p, h1–h6, section
`inline`	No — flows in text	No	span, a, strong, em
`inline-block`	No — flows in text	Yes	button (varies by browser)
`none`	—	Removed entirely	toggled / hidden elements

WHAT WE COVERED TODAY

- Every element is a **box**
- Four layers: content · padding · border · margin
- Default *content-box* adds padding+border on top of width
- Always start with `box-sizing: border-box`
- Units: `px` · `rem` · `em` · `%` · `vh/vw`
- Prefer `rem` for font sizes and spacing
- Display: `block` · `inline` · `inline-block` · `none`
- Display controls how elements flow in layout